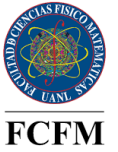




UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN
FACULTAD DE CIENCIAS FISICO-MATEMATICAS



Laboratorio Programación Orientada a Objetos

Práctica #3 - Encapsulamiento y Modificadores de
acceso

Profesor: Jorge Alberto Islas Pineda

Alumno: Jorge Jared Ramírez González

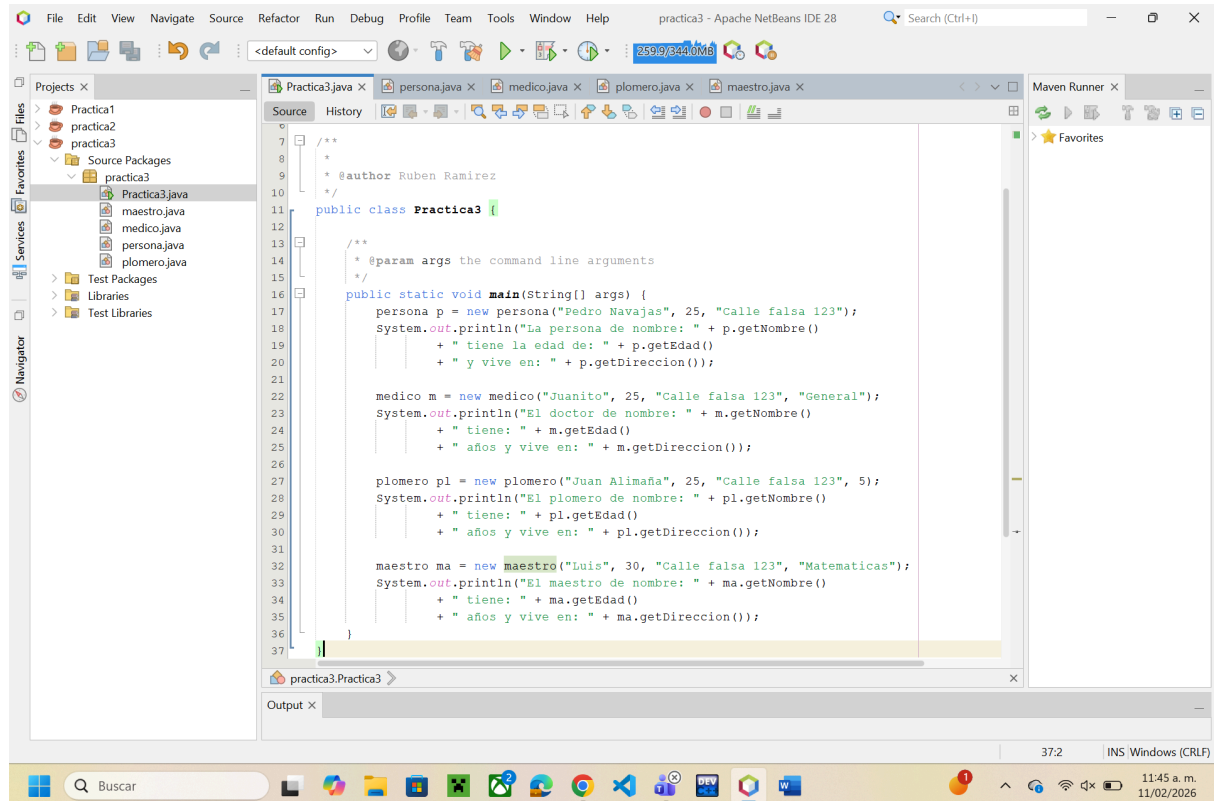
Matrícula: 2055454

Grupo: 032

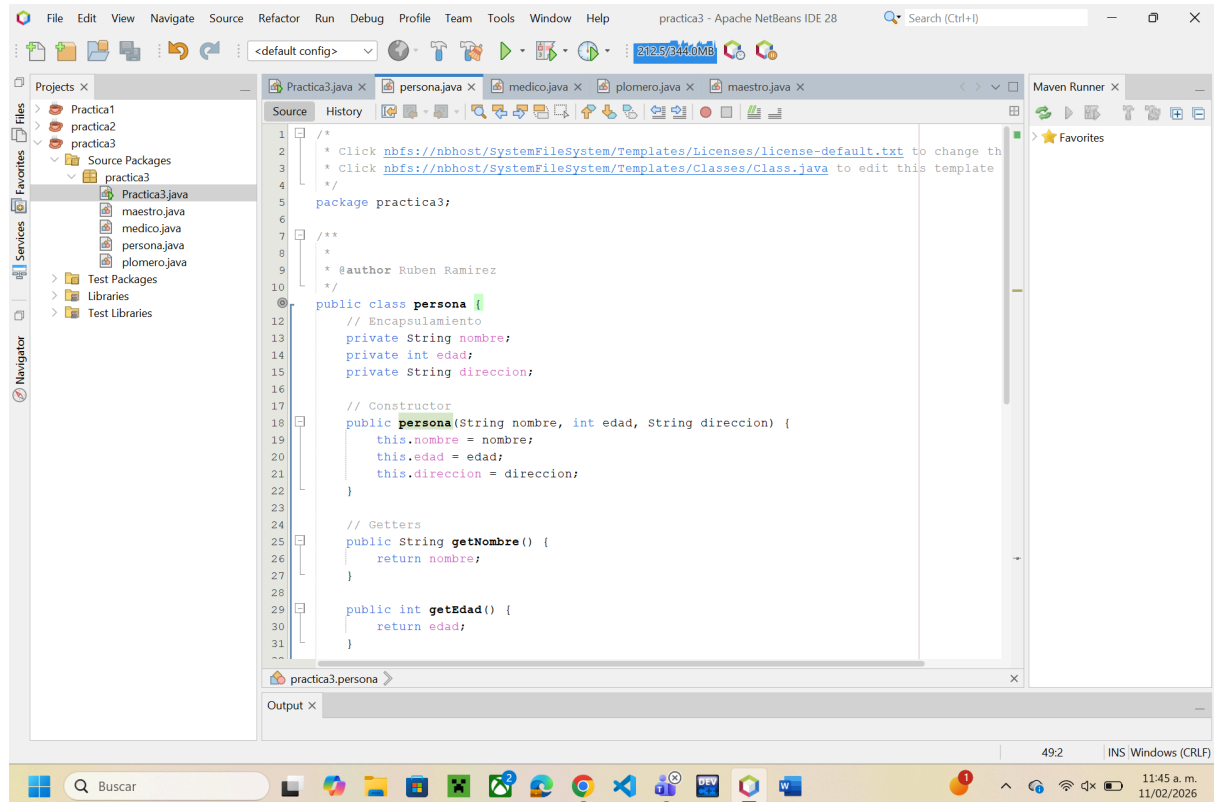
Ciudad Universitaria 13 de febrero 2026

Práctica #3: Encapsulamiento y modificadores de acceso.

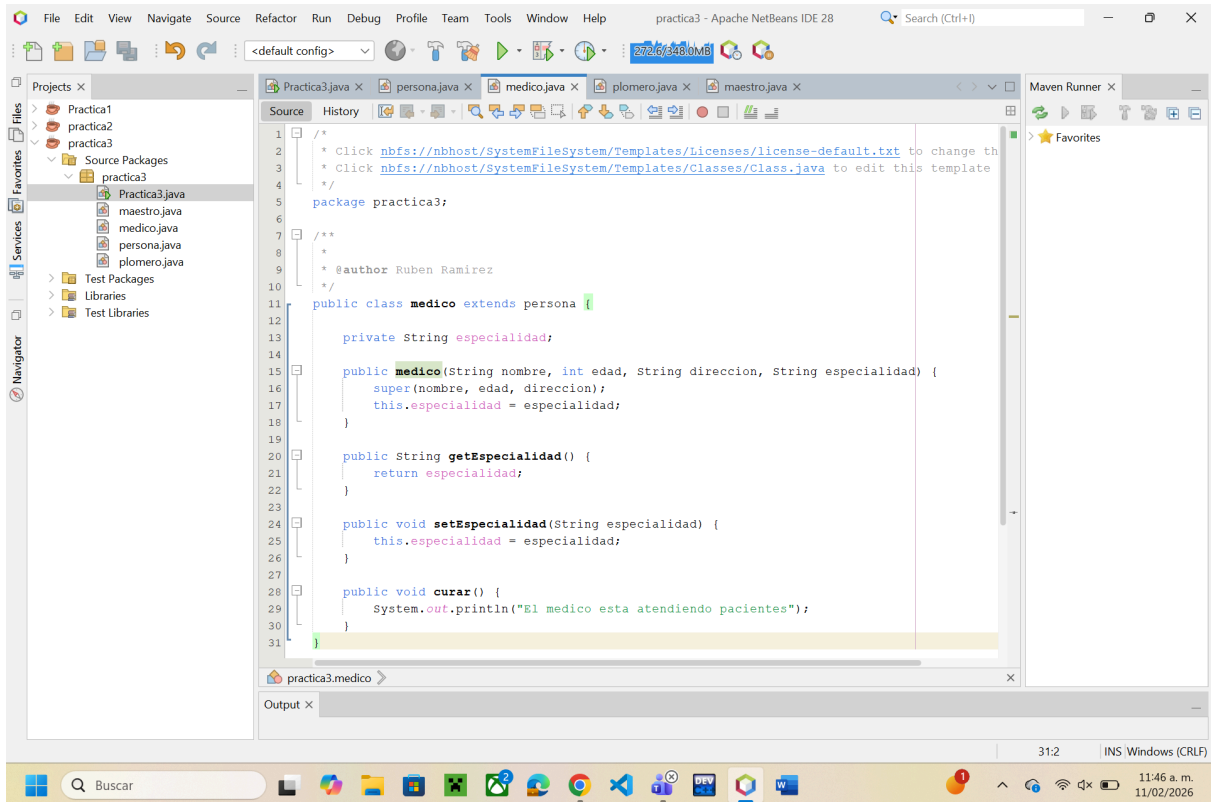
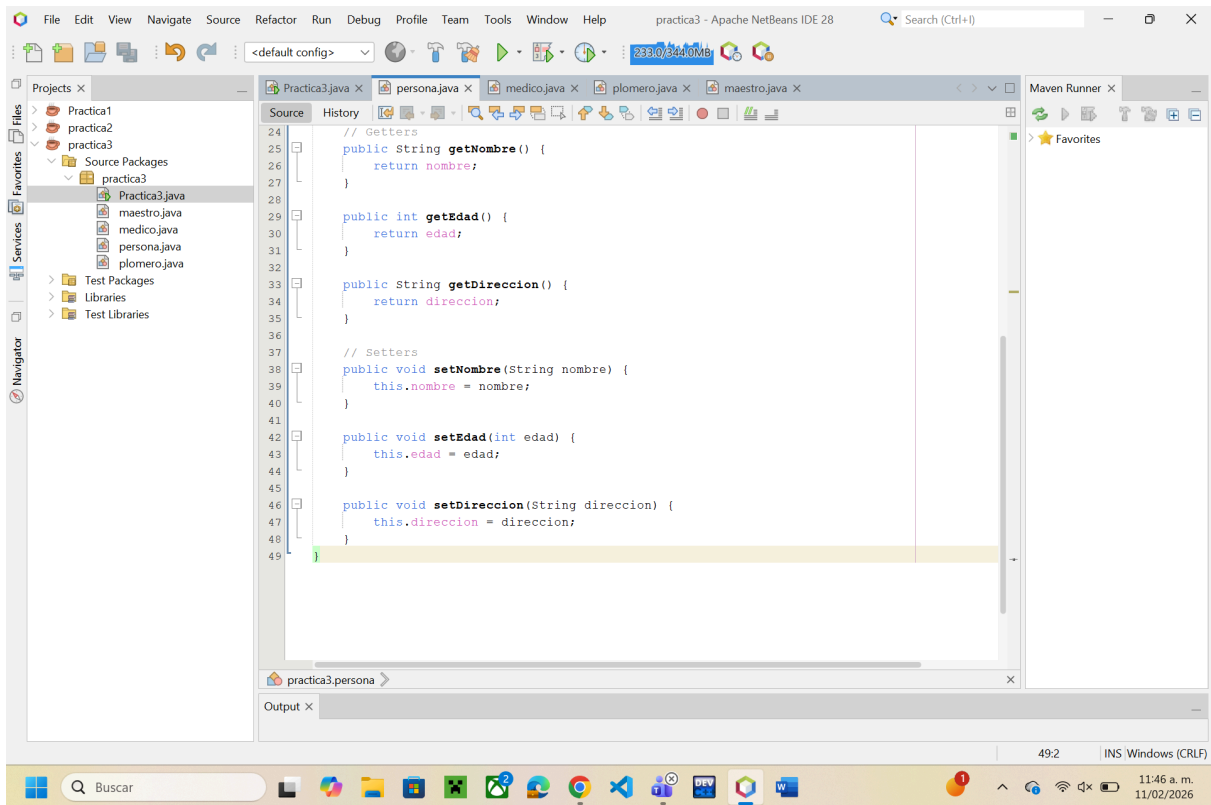
Código.

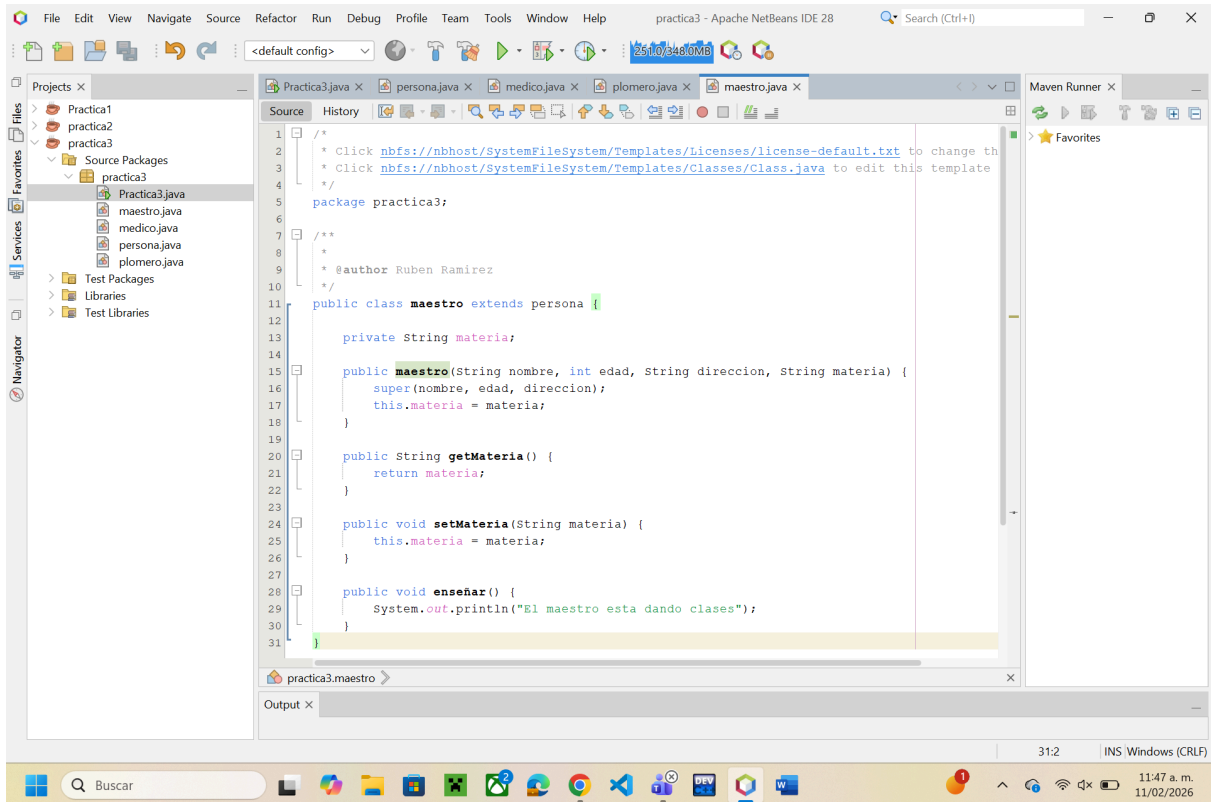
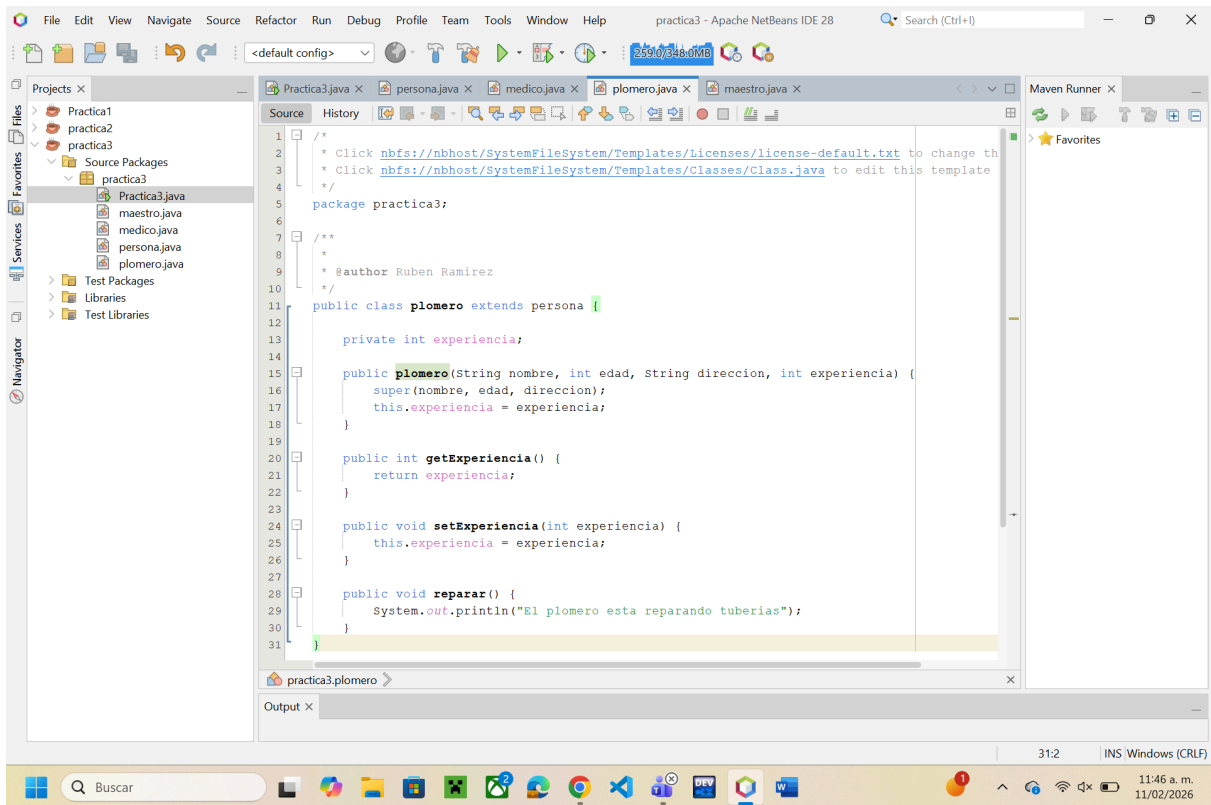


```
1  /**
2   *
3   * @author Ruben Ramirez
4   */
5  public class Practica3 {
6
7      /**
8       * @param args the command line arguments
9       */
10     public static void main(String[] args) {
11         persona p = new persona("Pedro Navañas", 25, "Calle falsa 123");
12         System.out.println("La persona de nombre: " + p.getNombre()
13             + " tiene la edad de: " + p.getEdad()
14             + " y vive en: " + p.getDireccion());
15
16         medico m = new medico("Juanito", 25, "Calle falsa 123", "General");
17         System.out.println("El doctor de nombre: " + m.getNombre()
18             + " tiene: " + m.getEdad()
19             + " años y vive en: " + m.getDireccion());
20
21         plomero pl = new plomero("Juan Alimaña", 25, "Calle falsa 123", 5);
22         System.out.println("El plomero de nombre: " + pl.getNombre()
23             + " tiene: " + pl.getEdad()
24             + " años y vive en: " + pl.getDireccion());
25
26         maestro ma = new maestro("Luis", 30, "Calle falsa 123", "Matematicas");
27         System.out.println("El maestro de nombre: " + ma.getNombre()
28             + " tiene: " + ma.getEdad()
29             + " años y vive en: " + ma.getDireccion());
30     }
31 }
```

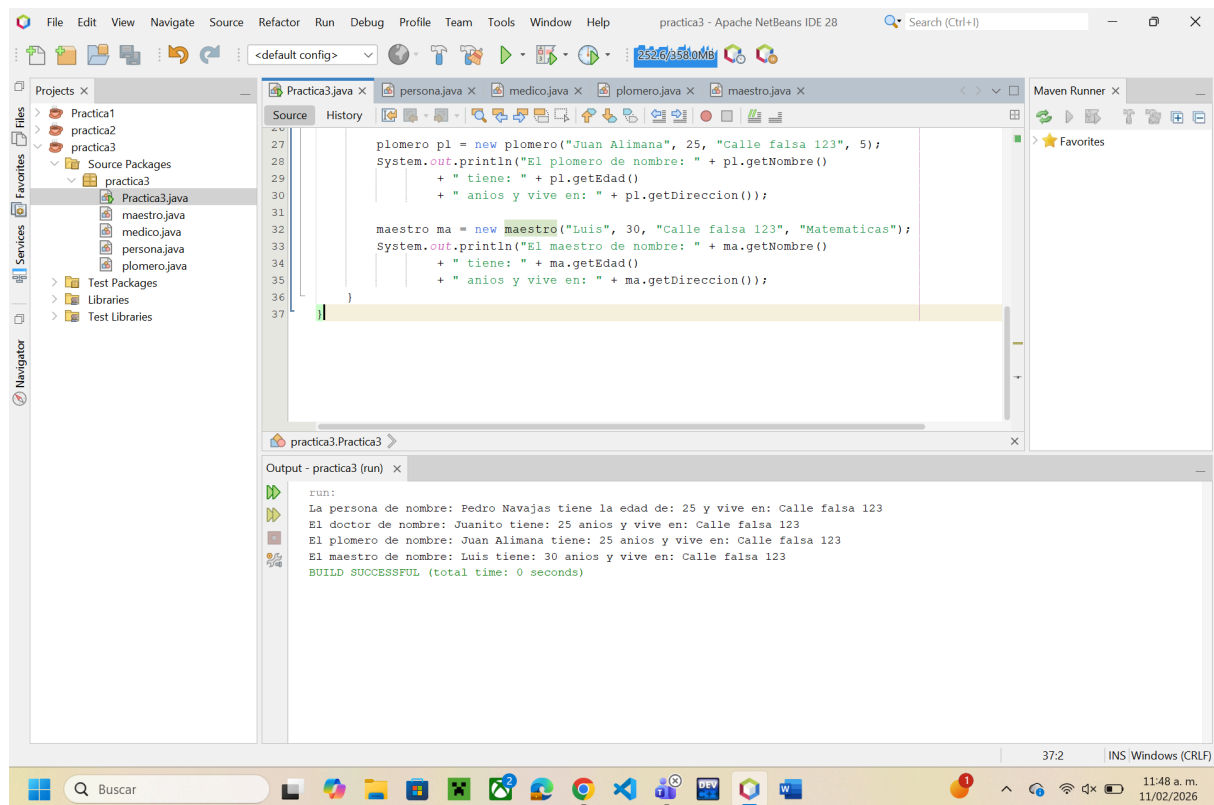


```
1  /**
2   * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change the license
3   * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
4   */
5  package practica3;
6
7  /**
8   *
9   * @author Ruben Ramirez
10  */
11  public class persona {
12      // Encapsulamiento
13      private String nombre;
14      private int edad;
15      private String direccion;
16
17      // Constructor
18      public persona(String nombre, int edad, String direccion) {
19          this.nombre = nombre;
20          this.edad = edad;
21          this.direccion = direccion;
22      }
23
24      // Getters
25      public String getNombre() {
26          return nombre;
27      }
28
29      public int getEdad() {
30          return edad;
31      }
32  }
```





Output del código.



Investigación: UML y Patrones de Diseño

¿Qué es UML?

UML significa **Unified Modeling Language** (Lenguaje Unificado de Modelado). Es un lenguaje visual que se utiliza para **analizar, diseñar y documentar** sistemas de software antes de programarlos.

Permite representar gráficamente elementos como:

- Clases
- Objetos
- Atributos
- Métodos
- Relaciones entre clases

Gracias a UML, los desarrolladores pueden entender mejor la estructura del sistema, detectar errores de diseño y comunicar ideas de manera más clara dentro de un equipo de trabajo.

Diagramas más utilizados en UML

♦ Diagrama de clases

Es el más común en Programación Orientada a Objetos.

Muestra:

- Nombre de la clase
- Atributos
- Métodos
- Relaciones como herencia o asociación

Ejemplo: una clase Persona puede ser la base de Médico, Maestro o Plomero.

♦ Diagrama de casos de uso

Describe las interacciones entre los usuarios y el sistema, mostrando qué acciones puede realizar cada actor.

♦ Diagrama de secuencia

Representa el orden en el que los objetos interactúan entre sí a lo largo del tiempo.

Importancia de UML

UML ayuda a:

- Planear antes de programar
- Reducir errores
- Mejorar la comunicación del equipo
- Documentar el sistema
- Facilitar el mantenimiento del software

¿Qué son los Patrones de Diseño?

Los patrones de diseño son soluciones probadas y reutilizables a problemas comunes que aparecen frecuentemente al desarrollar software.

No son código específico, sino formas recomendadas de estructurar clases y objetos para lograr programas más organizados, reutilizables y fáciles de mantener.

Tipos de patrones de diseño

♦ Creacionales

Ayudan a la creación de objetos.

Ejemplo:

- **Singleton** → garantiza que solo exista una instancia de una clase.

♦ Estructurales

Se enfocan en cómo se organizan las clases y objetos.

Ejemplo:

- **Adapter** → permite que clases con interfaces diferentes trabajen juntas.

♦ De comportamiento

Definen cómo interactúan y se comunican los objetos.

Ejemplo:

- **Observer** → cuando un objeto cambia, notifica a otros.

Importancia de los patrones de diseño

Permiten:

- Reutilizar soluciones
- Ahorrar tiempo
- Mantener código limpio
- Facilitar cambios futuros
- Mejorar la arquitectura del sistema

Relación con el encapsulamiento

El uso de modificadores como `private`, junto con `getters` y `setters`, forma parte de buenas prácticas de diseño, ya que protege la información interna de las clases y permite controlar cómo se accede a los datos.